ORF307_HW1_soln

January 25, 2025

ORF 307 HW1 soln

Q1 Interpolation of rational functions

For each interpolation condition, we multiply by the denominator of the right hand side. Then we reach the equation

$$f(t_i) = y_i = c_1 + c_2 t_i + c_3 t_i^2 - d_1 t_i y_i - d_2 t_i^2 y_i$$

We have K of these equations. Since they all share the same variables, c_1, c_2, c_3, d_1, d_2, we can rewrite it into a linear system, $A\theta = b$

$$A = \begin{bmatrix} 1 & t_1 & t_1^2 & -t_1 y_1 & -t_1^2 y_1 \\ 1 & t_2 & t_2^2 & -t_2 y_2 & -t_2^2 y_2 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & t_K & t_K^2 & -t_K y_K & -t_K^2 y_K \end{bmatrix}$$

$$b = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_K \end{bmatrix}$$

$A \in \mathbb{R}^{K,5}$ and $b \in \mathbb{R}^{K,1}$

Q2 Python timing test for linear equations

(a)

[1]:
```
import numpy as np
import scipy.linalg as LA
import time
N = 2000
A = np.random.normal(size=(N,N))
b = np.random.normal(size = N)
t0 = time.time()
Ainv = np.linalg.inv(A)

### some other operation to solve the system
x = Ainv @ b
```

```
t1 = time.time()
print('time to solve (a):', t1 - t0)
print('norm diff', np.linalg.norm(A @ x - b))
```

```
time to solve (a): 0.5517106056213379
norm diff 9.386687148739463e-11
```

(b)

[2]:
```
t2 = time.time()
x2 = np.linalg.solve(A,b)
t3 = time.time()
print('time to solve (b):', t3 - t2)
print('norm diff', np.linalg.norm(A @ x2 - b))
```

```
time to solve (b): 0.2420945167541504
norm diff 3.840756786501658e-11
```

(c)

[3]:
```
def forward_substitution(L, b):
    n = L.shape[0]
    x = np.zeros(n)
    for i in range(n):
        x[i] = (b[i] - L[i,:i] @ x[:i])/L[i, i]
    return x

def backward_substitution(U, b):
    n = U.shape[0]
    x = np.zeros(n)
    for i in reversed(range(n)):
        x[i] = (b[i] - U[i,i+1:] @ x[i+1:])/U[i, i]
    return x
```

[4]:
```
def solve_via_lu(P, L, U, b):
    '''
    Solve linear system Ax = b where
        A = PLU
    '''

    x1 = P.T @ b

    # now LUx = x1
    x2 = forward_substitution(L, x1)


    # noow Ux = x2
    x = backward_substitution(U, x2)
```

```
    return x
```

[5]:
```
from scipy.linalg import lu
t4 = time.time()
P, L, U = lu(A)  # Factor matrix A as A = PLU
x_lu = solve_via_lu(P, L, U, b)  # Solve via LU decomposition

t5 = time.time()
print('time to solve (c):', t5 - t4)
print('norm diff', np.linalg.norm(A @ x_lu - b))
```

```
time to solve (c): 0.27106451988220215
norm diff 3.302492569016002e-11
```

(d) Part (a) takes much longer than part (b) because we are inverting a matrix. Inverting a matrix takes up the vast majority of the time in part (a). Numpy's built-in solve function is much more efficient. Part (c) is faster than part (a) for similar reasons. However, it is still a bit slower than part (b). In general, built-in functions are very fast and they are tough to beat. Perhaps if our LU factorization method was coded in a lower level language such as C or C++ it would be faster.

Q3

To visualize what happens in each of the operations from parts (a) to (e), we can let $x = (1, 1, 0, 0)$ and $y = (1, 0, 1, 0)$, which covers all different combinations of symptoms. Note that $x_i y_i = 1$ if and only if both Alice and Bob have symptom i. Otherwise, $x_i y_i = 0$.

(a) $x^\top y$ is number of symptoms Alice and Bob have in common
True $x^T y = x_1 y_1 + \cdots + x_n y_n$. Each part of the sum will only be one if both Alice and Bob have symptoms.

(b) $\|x\|^2$ is the number of symptoms Alice has
True $\|x\|^2 = x_1^2 + \cdots + x_n^2 = x_1 + \cdots + x_n$. The last equality comes from each entry either being zero or one.

(c) $\mathbf{1}^\top y$ is number of symptoms Bob has
True $\mathbf{1}^\top y = y_1 + \cdots + y_n$

(d) $\|x - y\|^2$ is number of symptoms Alice has but Bob does not
False If $x = 0$ and $y = 1$ then there are zero symptoms Alice has and Bob doesn't but the quantity is one.

(e) $\mathbf{1}^\top (x - y)$ is the number of symptoms Alice has but Bob does not
False If $x = 0$ and $y = 1$ then this quantity will be negative which is not possible.

(f) $x \perp y$ means that Alice and Bob do not share any symptoms
True $x \perp y \implies x^\top y = 0 \implies x_i y_i = 0, \forall i \implies$ at least one of x_i or y_i is zero.